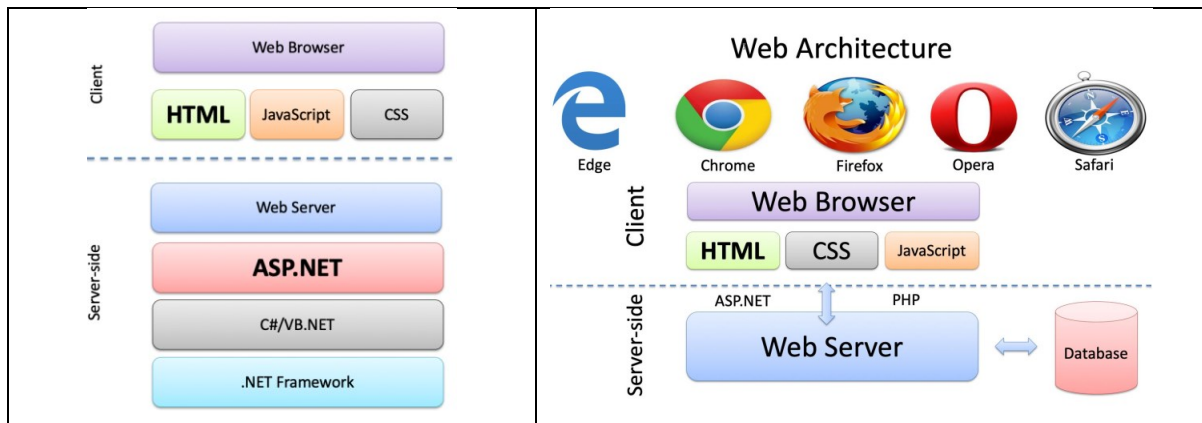


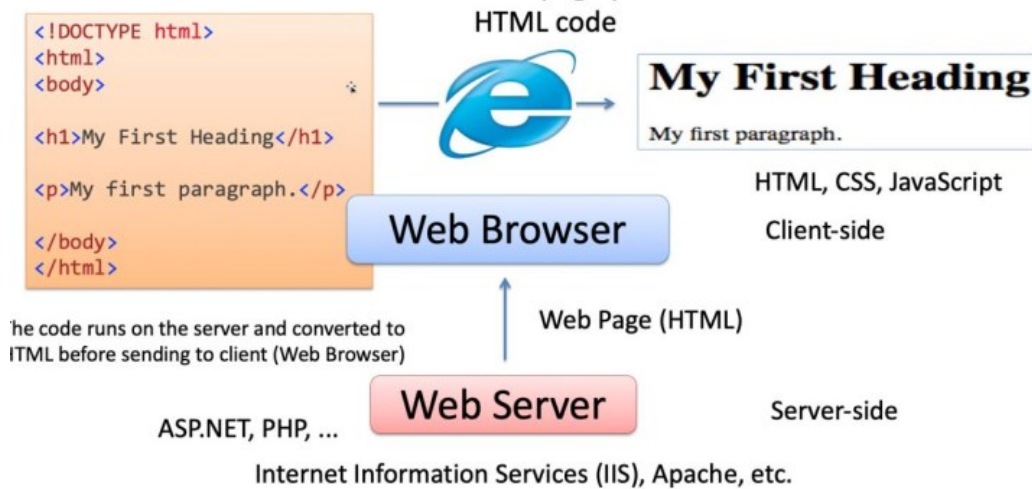
Contents

1. ASP Introduction	1
2. ASP.NET Core Fundamentals	4
3. ASP.NET Core Razor	5
4. HTML Forms	5
5. Web Pages connects to Database	8

1. ASP Introduction



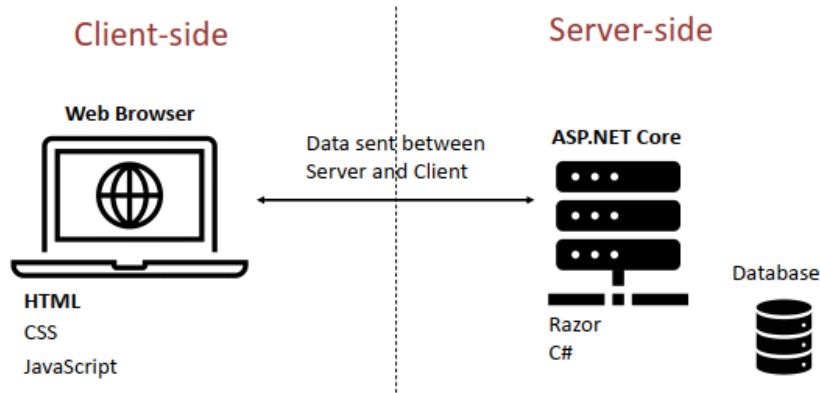
The Web Browser creates the visual web page you see in the browser based on the



ASP.NET Core is based on the .NET Core Framework (not the ordinary .NET Framework). We have:

- ASP.NET Core MVC (Model-View Controller). If you are familiar with the MVC approach, this could be your choice.
- **ASP.NET Core with Razor Pages**. This is the latest and recommended way. This has become the "default" approach for ASP.NET today. It mixes the best from all the others combined with PHP like syntax (PHP is probably the most popular Web Framework today)

ASP.NET Core Architecture

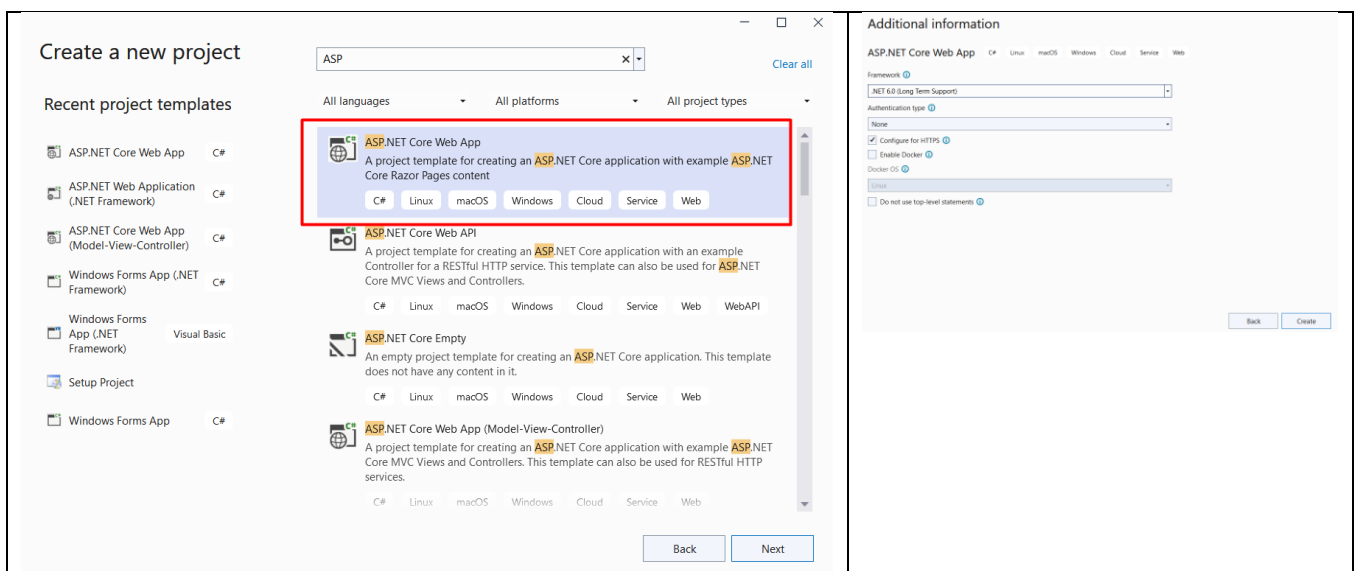


ASP.Net resource:

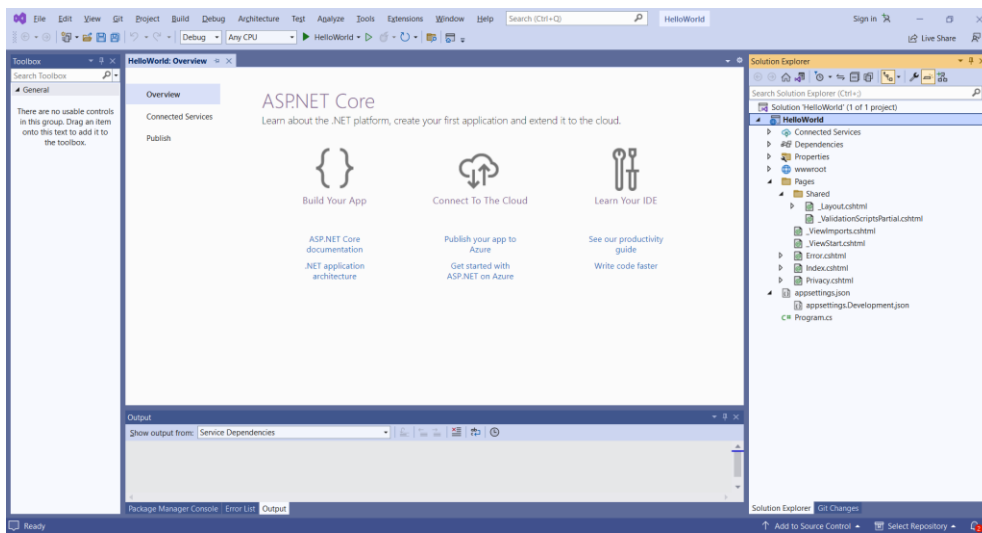
- [ASP.NET Core fundamentals overview | Microsoft Learn](#)
- [Introduction to Razor Pages in ASP.NET Core | Microsoft Learn](#)
- [Tutorial: Create a Razor Pages web app with ASP.NET Core | Microsoft Learn](#)
- [Razor syntax reference for ASP.NET Core | Microsoft Learn](#)

Different Programming models: 3 models

- **ASP.NET Web Forms**, • **ASP.NET MVC (Model-View-Controller)**;
- **ASP.NET Razor Pages**
 - Razor Single Page Model (comparable to PHP but using C# syntax instead)
 - Razor with Page Model (Code and Layout are separated in different Files)
 - Razor with MVC
 - **Different Types**
 - Razor Page (Single Page Model): Test1.cshtml
 - Razor Page (with Page Model): Test2.cshtml + Test2.cshtml.cs
 - Razor with MVC
 - **Each Razor page is a pair of files:**
 - **A .cshtml** file that contains HTML markup with C# code using Razor syntax.
 - **A .cshtml.cs** file that contains C# code that handles page events.



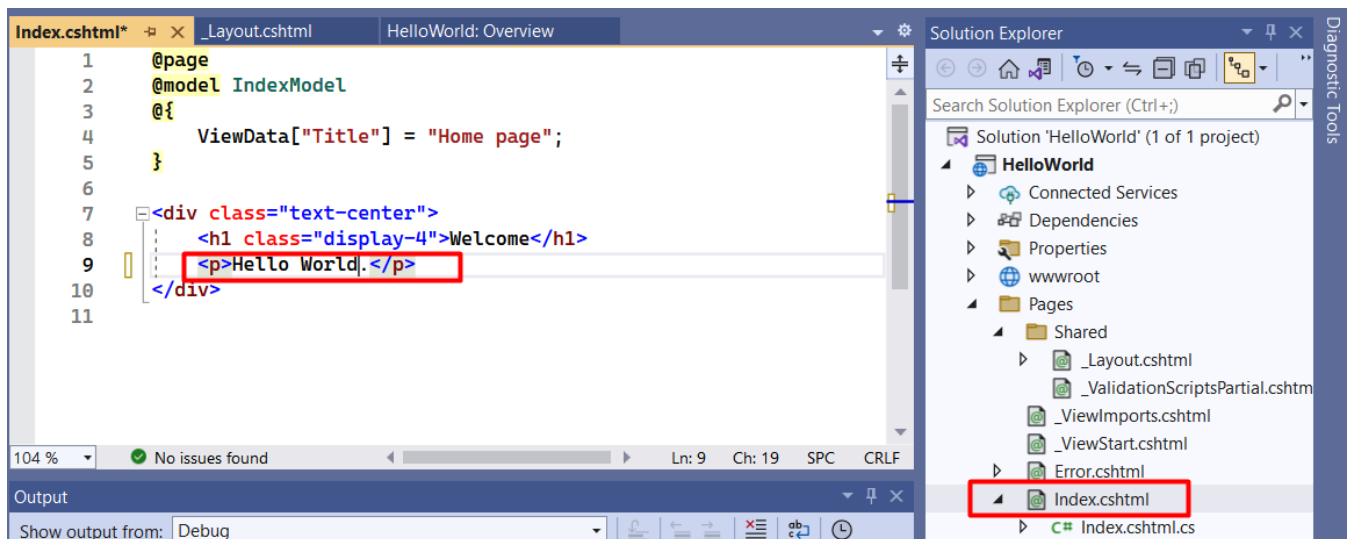
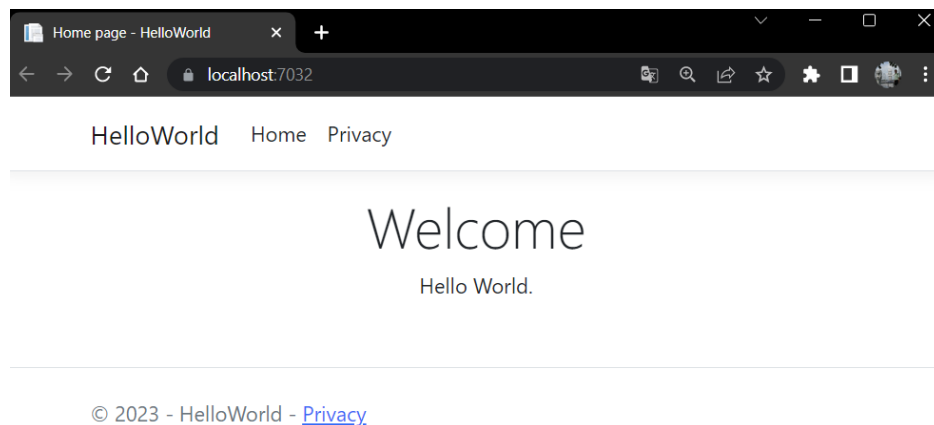
This tutorial will focus on this latest and newest approach, namely ASP.NET Core with Razor Pages.



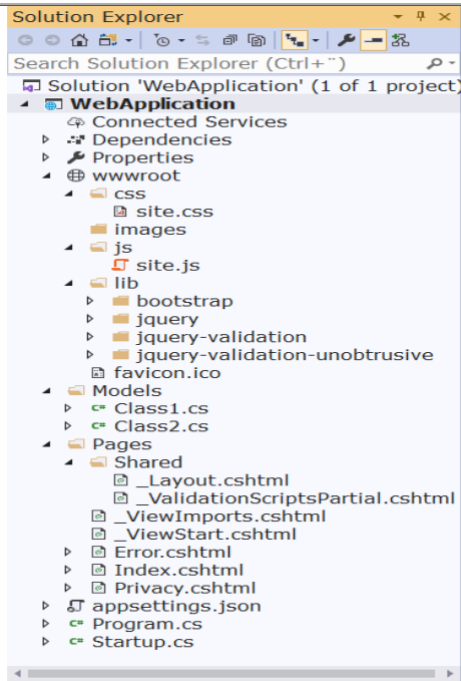
In addition, it is standard to have a folder called **"Models"**. This folder contains C# classes that takes care of the data. The data can, e.g., be a database or a file, e.g., a JSON file.

Moreover, we have what we call Supporting files. **Supporting files** have names that begin with an underscore (**_**)

- **_Layout.cshtml** file configures UI elements common to all pages. You can use this file to set up the navigation menu at the top of the page.



2. ASP.NET Core Fundamentals



- **appSettings.json**
 - Contains configuration data, such as connection strings
- **Program.cs**
 - Contains the entry point for the program
- **Startup.cs**
 - Contains code that configures app behavior.
- **wwwroot** folder
 - Contains static files, such as HTML files, JavaScript files, and CSS files
- **Pages** folder
 - You may put your ASP.NET web pages here
- **Models** folder
 - Here you create your Classe

Important Folders and Files:

- **appSettings.json** – This file contains configuration data, such as connection strings.
- **Program.cs** – This file contains the entry point for the program.
- **Startup.cs** - This file contains code that configures app behavior.
- **wwwroot** folder - Contains static files, such as Images, HTML files, JavaScript files, and CSS files.
- **Pages** folder – Here you are supposed to put your ASP.NET (".cshtml") web pages
- **Models** folder – Here you should create your Classes

appSettings.json

```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft": "Warning",
6       "Microsoft.Hosting.Lifetime": "Information"
7     }
8   },
9   "AllowedHosts": "*",
10
11   "ConnectionStrings": {
12     "ConnectionString": "DATA SOURCE=(local)\\SQL2017;UID=sa;PWD=SQL2017;DATABASE=LabDB"
13   }
14 }
```

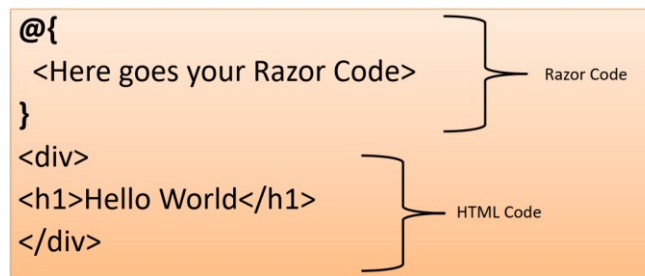
```
"ConnectionStrings": {
  "ConnectionString": "DATA SOURCE=(local)\\SQL2017;UID=sa;PWD=SQL2017;DATABASE=LabDB"
}
```

3. ASP.NET Core Razor

- Razor is a markup syntax for embedding server-based code into webpages.
- The Razor syntax consists of Razor markup, C#, and HTML. Files containing Razor generally have a .cshtml file extension.
- The default Razor language is HTML. Rendering HTML from Razor markup is no different than rendering HTML from an HTML file. HTML markup in .cshtml Razor files is rendered by the server unchanged.

Different Types

- Razor Page (Single Page Model): Test1.cshtml
- Razor Page (with Page Model): Test2.cshtml + Test2.cshtml.cs
 - Each Razor page is a pair of files:
 - A .cshtml file that contains HTML markup with C# code using Razor syntax.
 - A .cshtml.cs file that contains C# code that handles page events.
 - Razor Syntax
 - Razor supports C# and uses the @ symbol to transition from HTML to C#.
 - Razor evaluates C# expressions and renders them in the HTML output.
 - All code blocks must appear within @{ ... } brackets.



4. HTML Forms

The HTML <form> element is used to create a form on a web page.

- HTML Forms on a web page allows a user to enter data that is sent to a server for processing
- HTML Forms can contain Textboxes, Buttons, etc.
- The Form can have different attributes like method and action
- The method attribute determines the HTTP verb to use when the form is submitted, typically get or post

```
<form method="post">  
    <input name="FullName" type="text" />  
    <input id="OKButton" type="submit" value="OK" />  
</form>
```

- For the Form method we have the get or post attribute
- An HTML Form is used to collect user input. The user input is most often sent to a server for processing.

In ASP.NET you typically use the following Event Handlers:

- OnGet()
- OnPost()

So, in this Tutorial we will use those to react on Postbacks to the Server and handle data created on the Client.

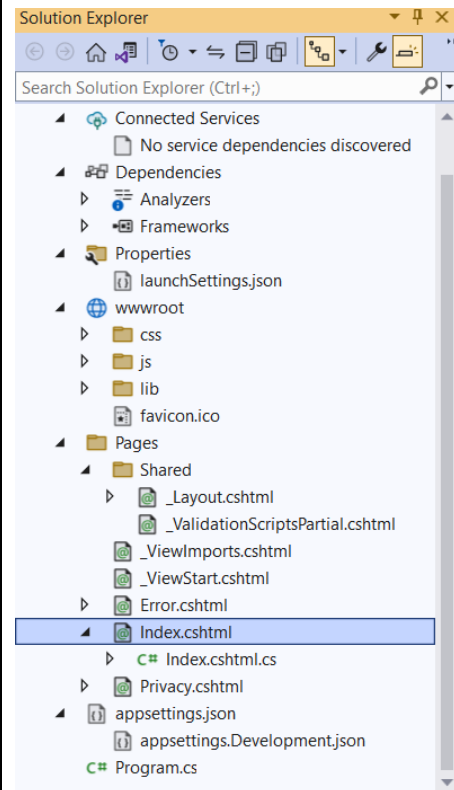
```

@page
@model IndexModel
@{
    ViewData["Title"] = "Home page";
}

<div class="text-center">
    <h1 class="display-4">Welcome</h1>
    <p>Hello World.</p>
</div>
<div>
<h1>Welcome</h1>
<p>Please enter your information:</p>
<form name="NameForm" id="NameForm" method="post">
    <label for="FullName">Name:</label>
    <input name="FullName" type="text" class="form-control input-lg" autofocus required />
    <br />
    <label for="Email">Email:</label>
    <input name="Email" type="email" class="form-control input-lg" required />
    <br />
    <input id="OKButton" type="submit" value="OK"
class="btn btn-success" />
</form>
<br />
<p>@ViewData["UserData"] </p>
</div>

```

Copy <div> .. </div> on the left to Index.cshtml



Data Binding: ASP.NET Core automatically assign the Form Data to these parameters

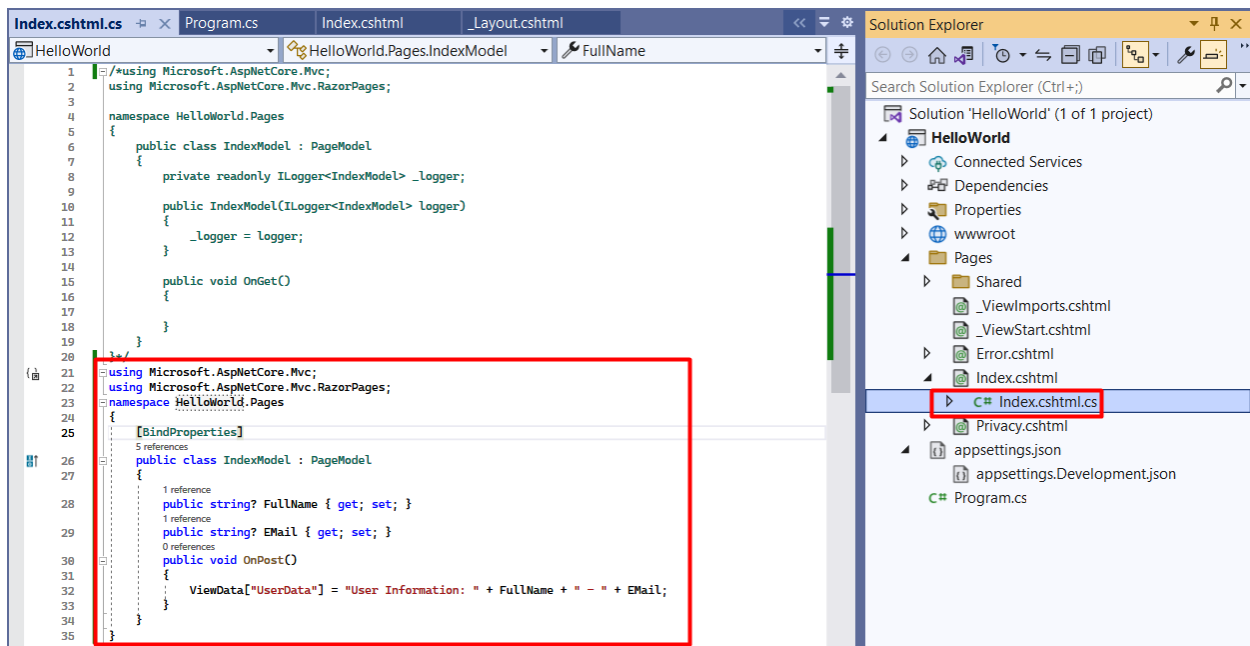
```

public void OnPost(string fullName, string eMail)
{
    ... use variables fullName and email
}

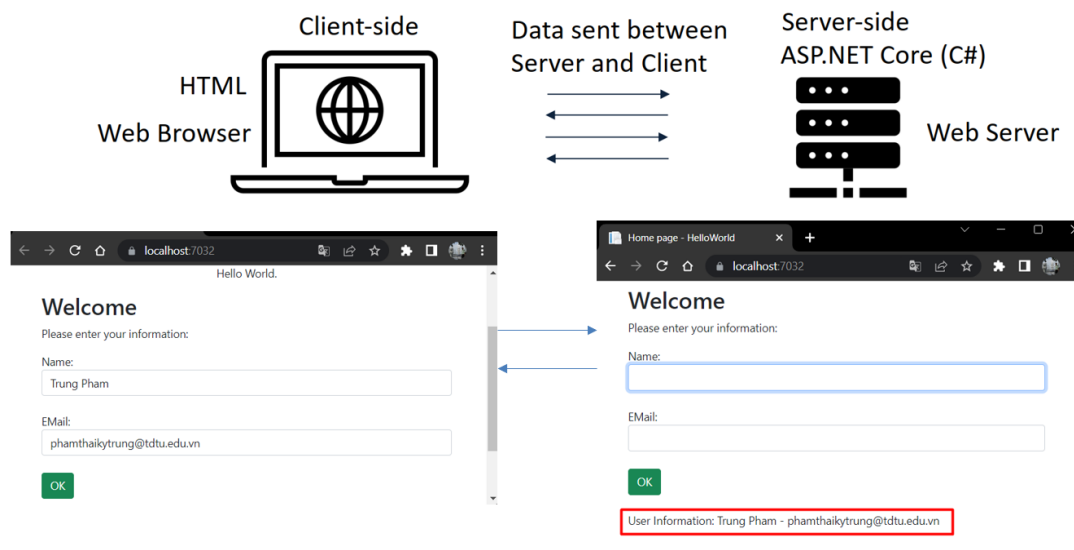
```

Now, refer **to Index.cshtml.cs**, try this code:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
namespace HelloWorld.Pages
{
    [BindProperties]
    public class IndexModel : PageModel
    {
        public string? FullName { get; set; }
        public string? EMail { get; set; }
        public void OnPost()
        {
            ViewData["UserData"] = "User Information: " + FullName + " - " + EMail;
        }
    }
}
```

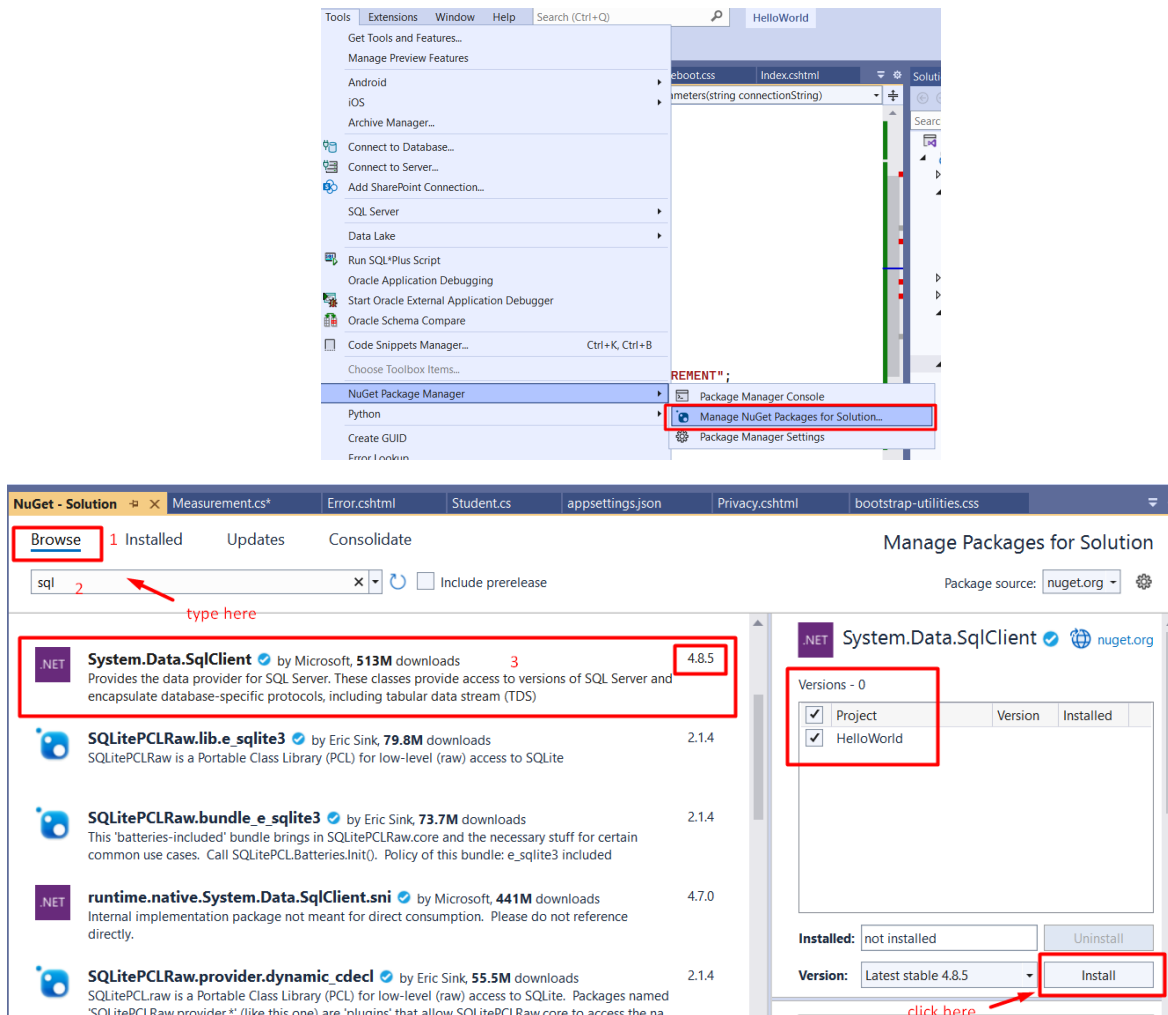


Output

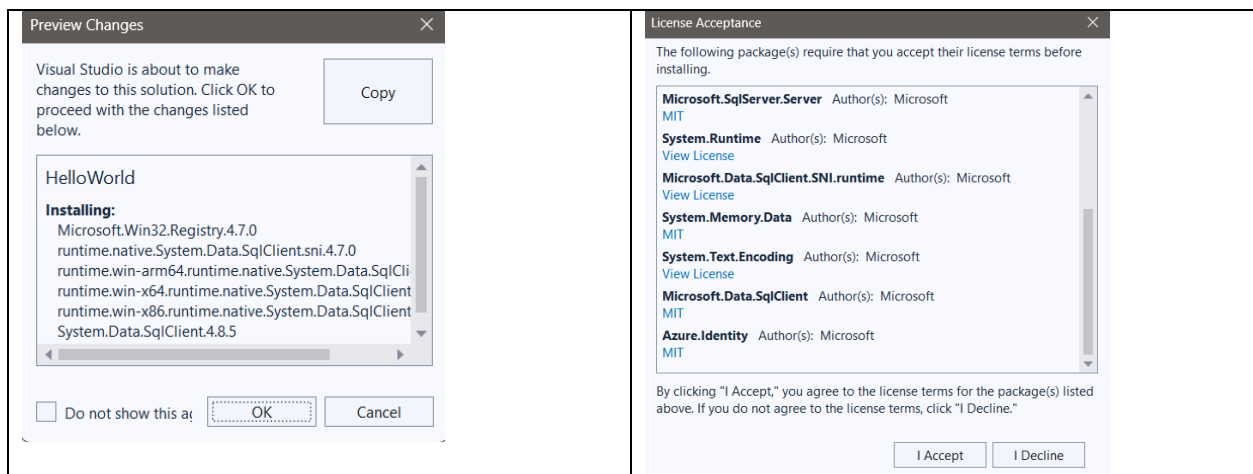


5. Web Pages connects to Database.

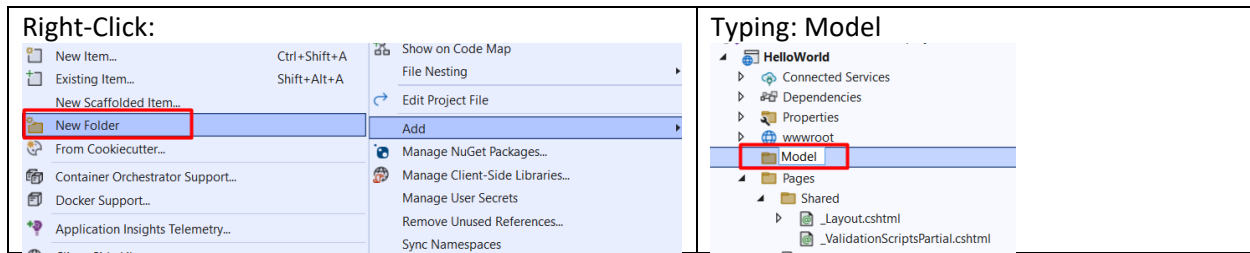
To connect MSSQL Database, we need to have **System.Data.SqlClient**, you use NuGet to install it



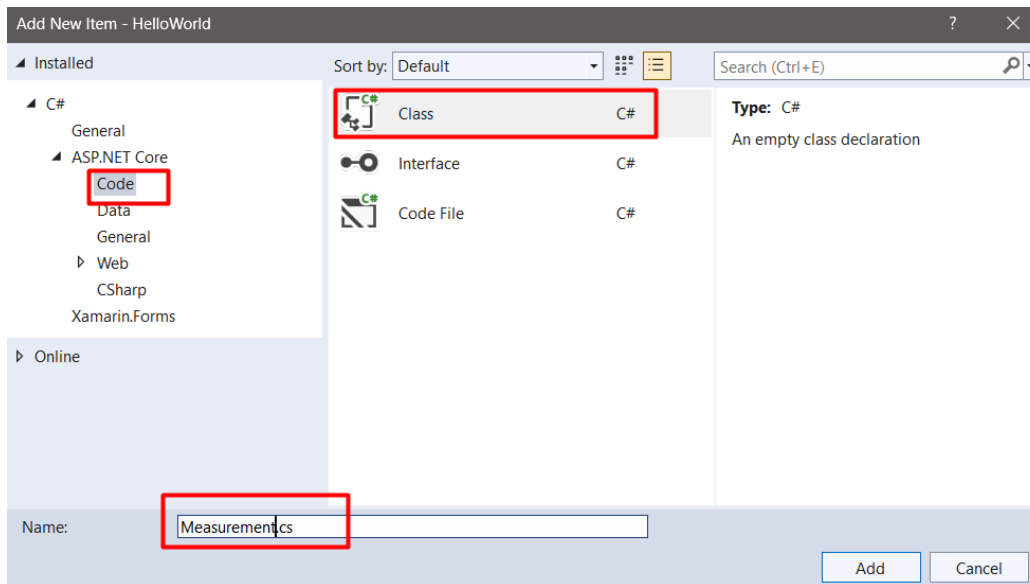
After click Install you need to click OK and I Accept.



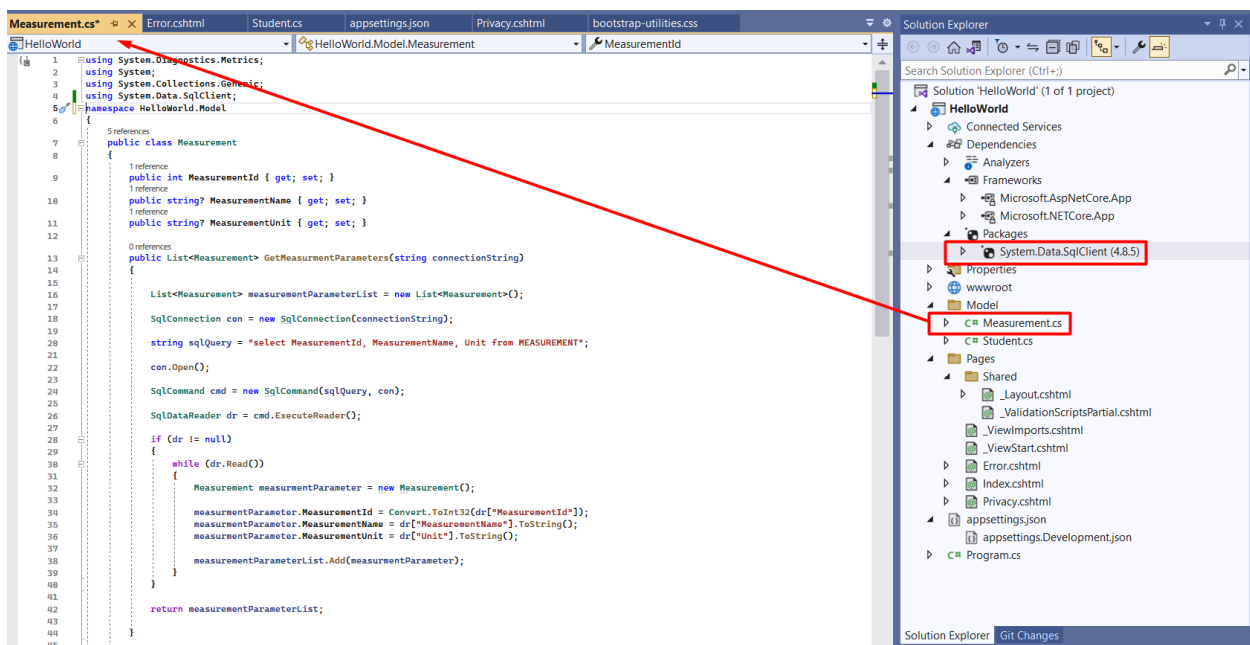
Create a Model folder on your Project by right-click:



Now you need to define a Measurement.cs (same name with data table)



Then you copy the code (Measurement.cs below), to open connection to DB and execute query command (*this seems to be like Windows Form*)



```

using System.Diagnostics.Metrics;
using System;
using System.Collections.Generic;
using System.Data.SqlClient;
namespace HelloWorld.Model
{
    public class Measurement
    {
        public int MeasurementId { get; set; }
        public string? MeasurementName { get; set; }
        public string? MeasurementUnit { get; set; }

        public List<Measurement> GetMeasurmentParameters(string connectionString)
        {
            List<Measurement> measurementParameterList = new List<Measurement>();

            SqlConnection con = new SqlConnection(connectionString);

            string sqlQuery = "select MeasurementId, MeasurementName, Unit from
MEASUREMENT";

            con.Open();

            SqlCommand cmd = new SqlCommand(sqlQuery, con);

            SqlDataReader dr = cmd.ExecuteReader();

            if (dr != null)
            {
                while (dr.Read())
                {
                    Measurement measurmentParameter = new Measurement();

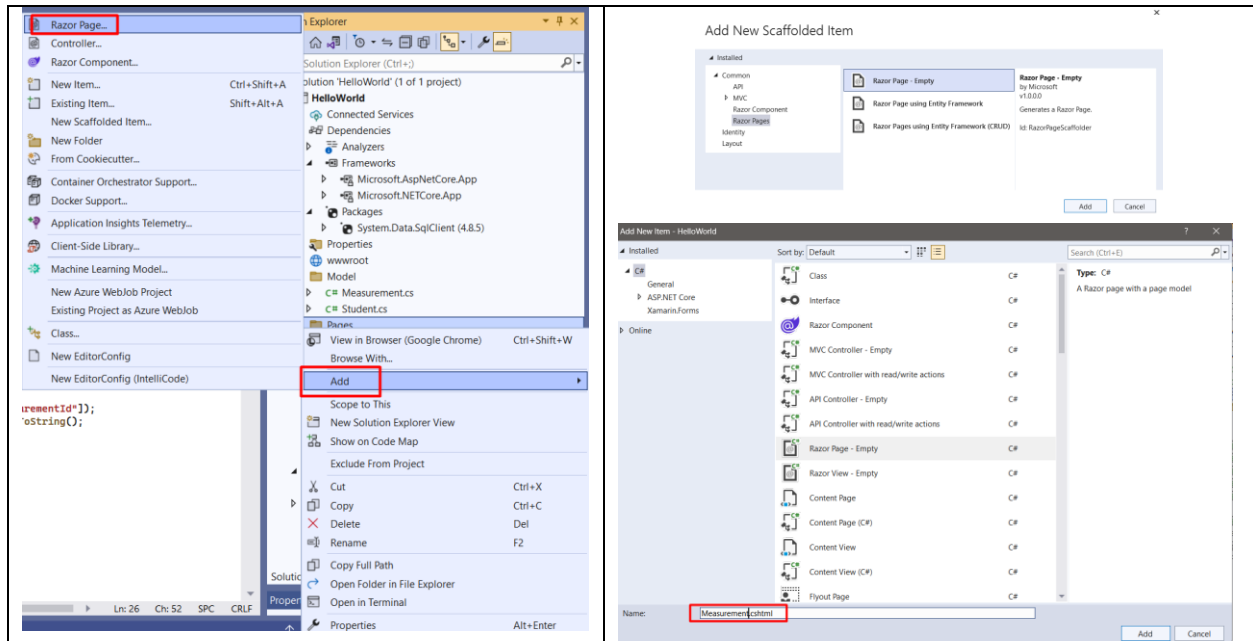
                    measurmentParameter.MeasurementId =
Convert.ToInt32(dr["MeasurementId"]);
                    measurmentParameter.MeasurementName =
dr["MeasurementName"].ToString();
                    measurmentParameter.MeasurementUnit = dr["Unit"].ToString();

                    measurementParameterList.Add(measurmentParameter);
                }
            }

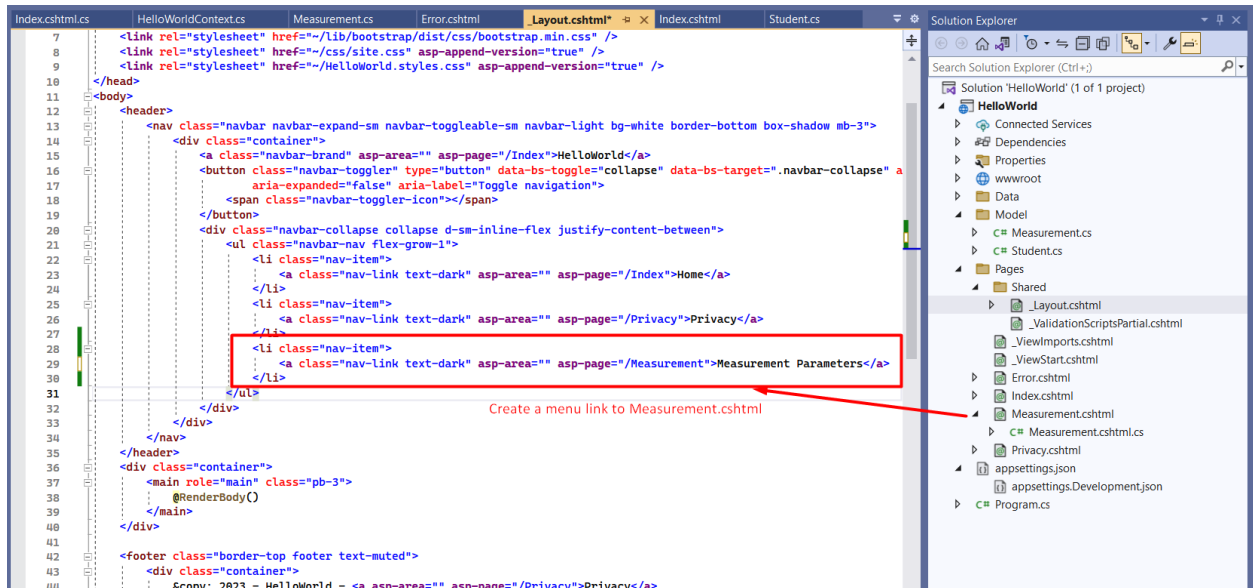
            return measurementParameterList;
        }
    }
}

```

Now you need to create a razor page named "Measurement.cshtml"

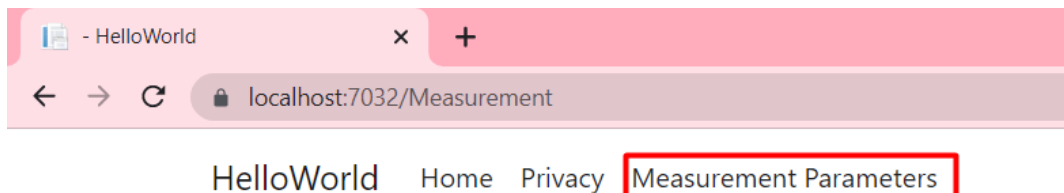


Create a menu link to Measurement.cshtml

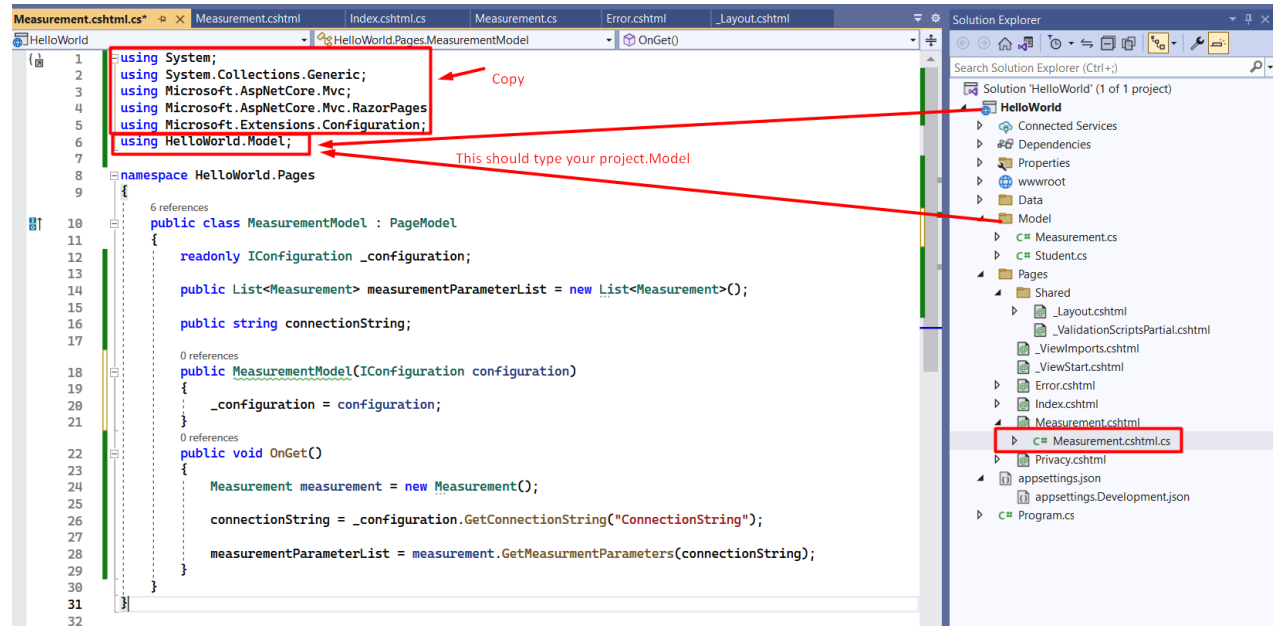


```
<li class="nav-item">
  <a class="nav-link text-dark" asp-area="" asp-page="/Measurement">Measurement Parameters</a>
</li>
```

Output



Add code to Measurement.cshhtml.cs



Measurement.cshhtml.cs

```
using System;
using System.Collections.Generic;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Configuration;
using HelloWorld.Model;

namespace HelloWorld.Pages
{
    public class MeasurementModel : PageModel
    {
        readonly IConfiguration _configuration;

        public List<Measurement> measurementParameterList = new List<Measurement>();

        public string connectionString;

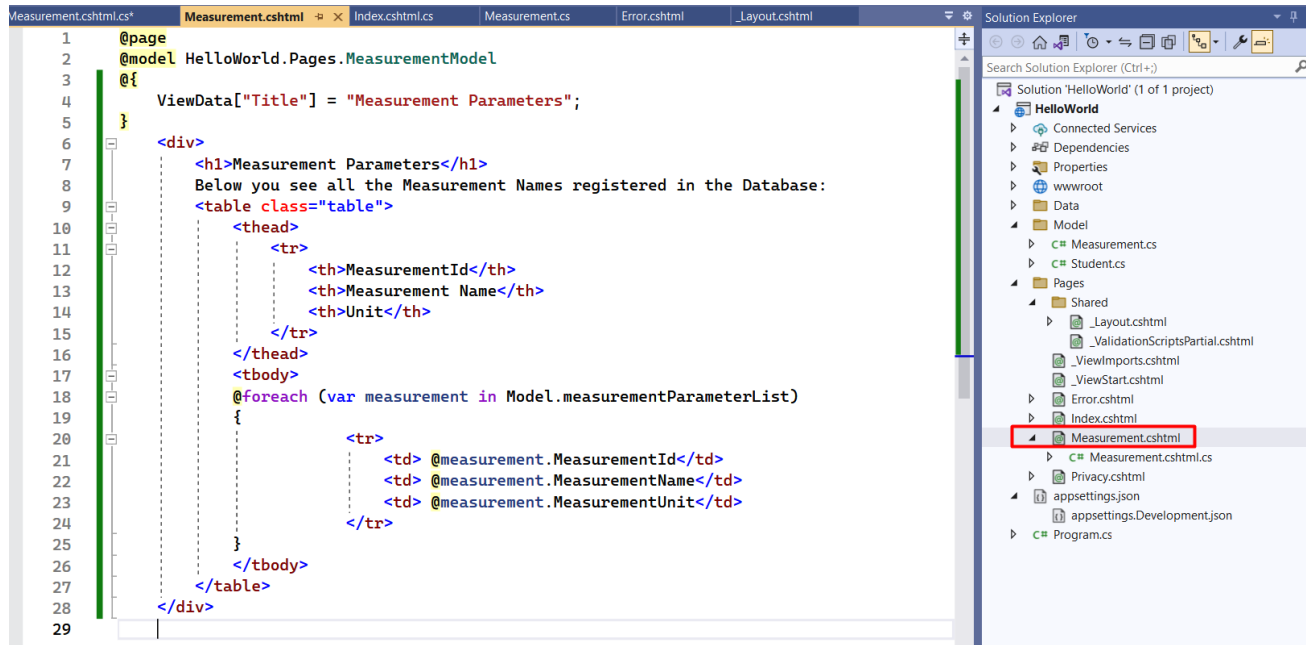
        public MeasurementModel(IConfiguration configuration)
        {
            _configuration = configuration;
        }

        public void OnGet()
        {
            Measurement measurement = new Measurement();

            connectionString = _configuration.GetConnectionString("ConnectionString");

            measurementParameterList = measurement.GetMeasurementParameters(connectionString);
        }
    }
}
```

Add code to Measurement.cshmtl

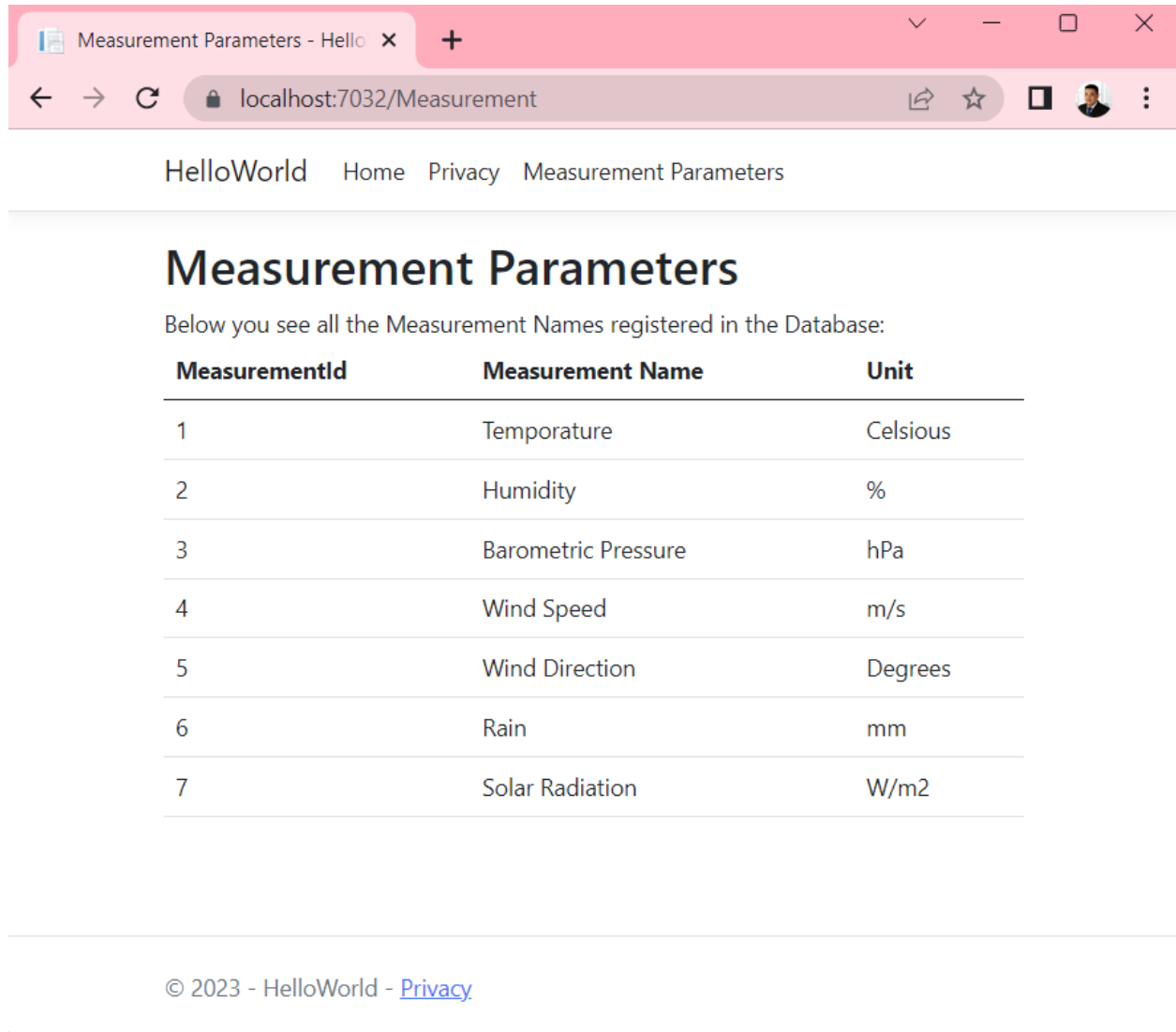


Measurement.cshmtl

```
@page
@model HelloWorld.Pages.MeasurementModel
@{
    ViewData["Title"] = "Measurement Parameters";
}

<div>
    <h1>Measurement Parameters</h1>
    Below you see all the Measurement Names registered in the Database:
    <table class="table">
        <thead>
            <tr>
                <th>MeasurementId</th>
                <th>Measurement Name</th>
                <th>Unit</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var measurement in Model.measurementParameterList)
            {
                <tr>
                    <td> @measurement.MeasurementId</td>
                    <td> @measurement.MeasurementName</td>
                    <td> @measurement.MeasurementUnit</td>
                </tr>
            }
        </tbody>
    </table>
</div>
```

Run the project:



Measurement Parameters - Hello X +

← → ↻ 🔒 localhost:7032/Measurement 🔗 ☆ 🗨 👤 ⋮

HelloWorld Home Privacy Measurement Parameters

Measurement Parameters

Below you see all the Measurement Names registered in the Database:

MeasurementId	Measurement Name	Unit
1	Temperature	Celsious
2	Humidity	%
3	Barometric Pressure	hPa
4	Wind Speed	m/s
5	Wind Direction	Degrees
6	Rain	mm
7	Solar Radiation	W/m2

© 2023 - HelloWorld - [Privacy](#)

Enjoy 😊

P/S: Refer to this Tutorial to conduct your Lab8-9-10.